Re-thinking Retail Inventory Taxonomy

Bharath C

2026-06-18

Contents

---

1. Overview

Retail businesses typically manage their product catalog in one of two ways. Some rely on **text taxonomy** — grouping everything labeled "BAG" or "MUG" under a single stocking policy. Others lean on **customer RFM segmentation** — identifying who spends the most and building supply decisions around those accounts.

Both approaches are useful in their own right, but they share a blind spot that becomes expensive at the warehouse level.

Consider two buyers who both rank as "high-value" under RFM. The first is a B2B wholesaler placing a single invoice for 2,000 cheap novelty bags at £0.87 each. The second is a boutique retailer ordering 2 premium leather items at £45 each. Their Monetary scores are similar. Their physical requirements are completely different — one needs a cross-docking bay and pallet space, the other needs a secure shelf and a VIP marketing tag. The RFM model treats them identically.

The same distortion exists in text-based catalogs. Applying a uniform reorder rule to everything labeled "BAG" will leave you chronically under-stocked on your revenue anchors and sitting on dead inventory that hasn't moved in months.

This analysis takes a different approach. Rather than aggregating by customer or by product name, we aggregate by **StockCode** — profiling each item on four commercial performance metrics: total units moved, gross revenue contributed, number of distinct buyers, and average unit price. K-Means clustering then finds the natural groupings in that space.

The result is five operationally distinct inventory archetypes. They don't come from what a product is called or who bought it — they come from how it actually behaves in the market. The rest of this report walks through how we got there.

---

2. Data & Initial Exploration

```
zip_file   <- "online_retail.zip"
excel_file <- "Online Retail.xlsx"

if (!file.exists(zip_file)) {
  download.file(
    "https://archive.ics.uci.edu/static/public/352/online+retail.zip",
    zip_file, mode = "wb"
  )
}
if (!file.exists(excel_file)) unzip(zip_file, excel_file)

retail_data <- read_excel(excel_file)
```

```
total_sale_value <- retail_data %>%
  summarise(Total = sum(Quantity * UnitPrice)) %>%
  pull(Total)

overall_summary <- retail_data %>%
  summarise(
    Distinct_Orders    = n_distinct(InvoiceNo),
    Distinct_Products  = n_distinct(StockCode),
```

```
    Distinct_Customers = n_distinct(CustomerID),
    Total_Sale_Value   = sum(Quantity * UnitPrice),
    AOV                = sum(Quantity * UnitPrice) / n_distinct(InvoiceNo)
  )

overall_summary
```

```
## # A tibble: 1 x 5
##   Distinct_Orders Distinct_Products Distinct_Customers Total_Sale_Value   AOV
##             <int>             <int>              <int>            <dbl> <dbl>
## 1           25900              4070               4373         9747748.  376.
```

The raw dataset covers **25,900 orders**, **4,070 product codes**, and a gross revenue of **£9,747,748**. The baseline AOV is £376.36 — but this figure includes cancellations and non-product adjustment rows, so it's only useful as a before/after reference point.

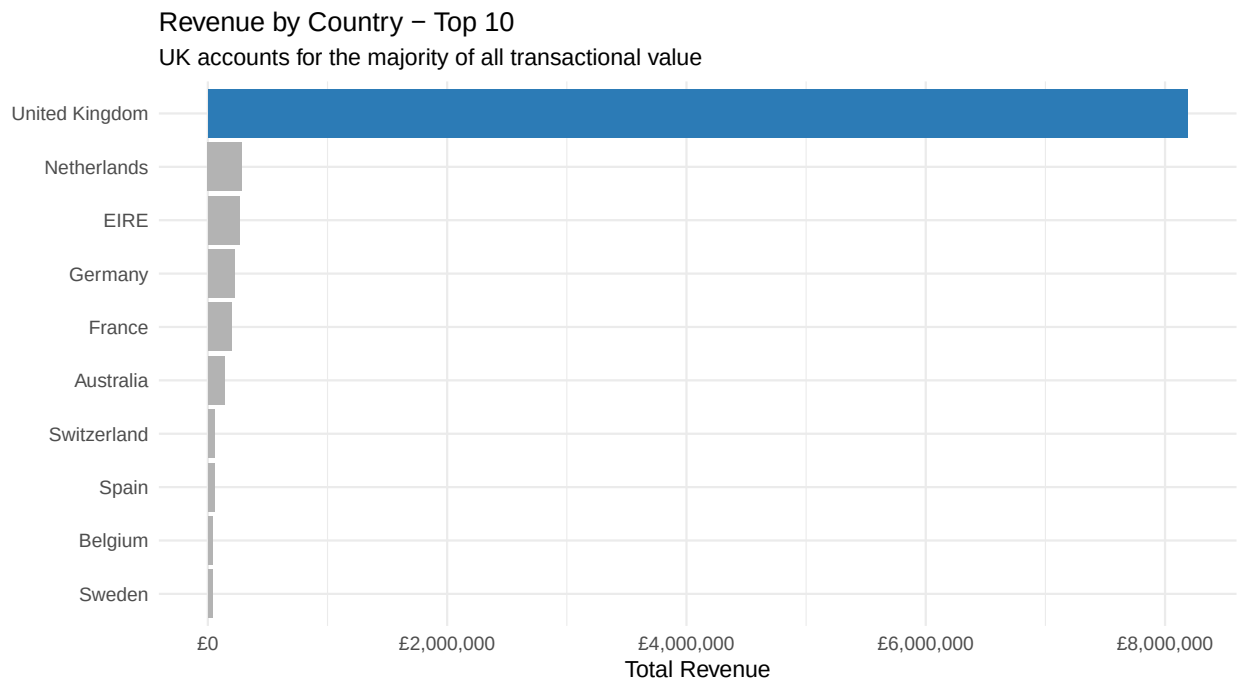
2.1 Geographic Concentration

```
sales_by_country <- retail_data %>%
  summarise(
    Orders       = n_distinct(InvoiceNo),
    Revenue      = sum(Quantity * UnitPrice),
    .by = Country
  ) %>%
  mutate(Revenue_Pct = round(100 * Revenue / sum(Revenue), 2)) %>%
  arrange(desc(Revenue))

sales_by_country %>% head(10)
```

```
## # A tibble: 10 x 4
##    Country        Orders  Revenue Revenue_Pct
##    <chr>           <int>    <dbl>       <dbl>
##  1 United Kingdom  23494 8187806.       84
##  2 Netherlands       101  284662.        2.92
##  3 EIRE              360  263277.        2.7
##  4 Germany           603  221698.        2.27
##  5 France            461  197404.        2.03
##  6 Australia          69  137077.        1.41
##  7 Switzerland        74   56385.        0.58
##  8 Spain             105   54775.        0.56
##  9 Belgium           119   40911.        0.42
## 10 Sweden             46   36596.        0.38
```

```
sales_by_country %>%
  slice_max(Revenue, n = 10) %>%
  mutate(Country = fct_reorder(Country, Revenue)) %>%
  ggplot(aes(x = Revenue, y = Country, fill = Country == "United Kingdom")) +
  geom_col(show.legend = FALSE) +
  scale_x_continuous(labels = label_comma(prefix = "£")) +
  scale_fill_manual(values = c("grey70", "#2c7bb6")) +
```

```
  labs(
    title = "Revenue by Country - Top 10",
    subtitle = "UK accounts for the majority of all transactional value",
    x = "Total Revenue", y = NULL
  ) +
  theme_minimal(base_size = 12)
```



The UK accounts for over **84% of all revenue**. Restricting the analysis to the UK eliminates cross-border logistics noise, currency variance, and customs-related return patterns.

2.2 Missing Customer IDs

```
missing_id_pct   <- 100 * mean(is.na(retail_data$CustomerID))

na_value <- retail_data %>%
  filter(is.na(CustomerID)) %>%
  summarise(Revenue = sum(Quantity * UnitPrice)) %>%
  pull(Revenue)

na_value_pct <- 100 * na_value / total_sale_value

tibble(
  Metric         = c("Rows missing CustomerID", "Revenue from anonymous rows"),
  Value          = c(paste0(round(missing_id_pct, 2), "%"), paste0(round(na_value_pct, 2), "%"))
)
```

```
## # A tibble: 2 x 2
##   Metric                      Value
##   <chr>                       <chr>
```

```
## 1 Rows missing CustomerID     24.93%
## 2 Revenue from anonymous rows 14.85%
```

About **25% of rows have no CustomerID** — these are likely anonymous guest checkouts. They're dropped for any customer-level work, but they still represent ~15% of total revenue. Worth flagging so the business doesn't treat the cleaned customer dataset as the full picture.

2.3 StockCode Structure

```
retail_data <- retail_data %>%
  mutate(
    nChar_StockCode = str_length(StockCode),
    nChar_InvoiceNo = str_length(InvoiceNo)
  )

retail_data %>%
  count(nChar_StockCode, sort = TRUE) %>%
  mutate(Pct = round(100 * n / sum(n), 2))
```

```
## # A tibble: 10 x 3
##    nChar_StockCode      n   Pct
##              <int>  <int> <dbl>
##  1               5 487036 89.9
##  2               6  51488  9.5
##  3               4   1276  0.24
##  4               1    715  0.13
##  5               3    710  0.13
##  6               7    390  0.07
##  7               2    144  0.03
##  8              12     71  0.01
##  9               9     48  0.01
## 10               8     31  0.01
```

Standard retail product codes are 5–7 characters long. Anything outside that range is an operational entry — postage charges, manual bank adjustments, or bulk delivery fees. These rows have no place in a product inventory model.

---

3. Data Cleaning

```
cleaned_retail <- retail_data %>%
  mutate(StockCode = tolower(StockCode)) %>%
  filter(Country == "United Kingdom") %>%
  filter(!is.na(CustomerID)) %>%
  filter(Quantity > 0, UnitPrice > 0) %>%
  filter(nChar_StockCode %in% c(5, 6, 7))

cat("Raw rows:    ", nrow(retail_data), "\n")
```

```
## Raw rows:     541909
```

```
cat("Cleaned rows:", nrow(cleaned_retail), "\n")
```

```
## Cleaned rows: 353985
```

Each filter step has a concrete rationale:

Filter	Why
`Country == "United Kingdom"`	UK is 84%+ of data; removes currency and logistics noise
`!is.na(CustomerID)`	Required for any customer or product tracking
`Quantity > 0, UnitPrice > 0`	Removes cancellation credits and zero-price adjustments
`nChar_StockCode %in% c(5,6,7)`	Keeps only physical retail merchandise

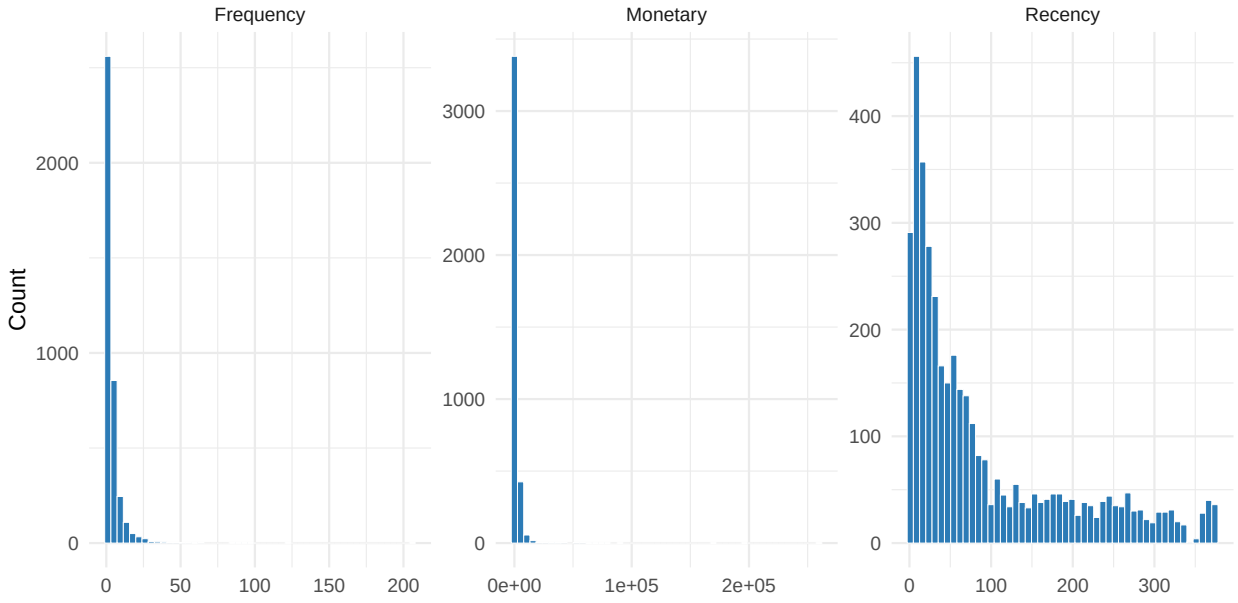
---

4. Model 1 — Customer RFM Segmentation

```
snapshot_date <- max(as_date(retail_data$InvoiceDate), na.rm = TRUE) + days(1)

rfm_data <- retail_data %>%
  mutate(
    StockCode       = tolower(StockCode),
    nChar_StockCode = str_length(StockCode)
  ) %>%
  filter(Country == "United Kingdom", !is.na(CustomerID),
         Quantity > 0, UnitPrice > 0,
         nChar_StockCode %in% c(5, 6, 7) | str_detect(StockCode, "dcgs")) %>%
  mutate(Line_Total = Quantity * UnitPrice) %>%
  group_by(CustomerID) %>%
  summarise(
    Recency   = as.numeric(snapshot_date - max(as_date(InvoiceDate))),
    Frequency = n_distinct(InvoiceNo),
    Monetary  = sum(Line_Total),
    .groups   = "drop"
  )
```

4.1 Distribution Check

```
rfm_data %>%
  pivot_longer(cols = c(Recency, Frequency, Monetary), names_to = "Metric", values_to = "Value") %>%
  ggplot(aes(x = Value)) +
  geom_histogram(bins = 50, fill = "#2c7bb6", colour = "white", linewidth = 0.2) +
  facet_wrap(~Metric, scales = "free") +
  labs(title = "Raw RFM Distributions - Heavy Right Skew",
       subtitle = "A log transform is needed before feeding these into K-Means",
       x = NULL, y = "Count") +
  theme_minimal(base_size = 12)
```

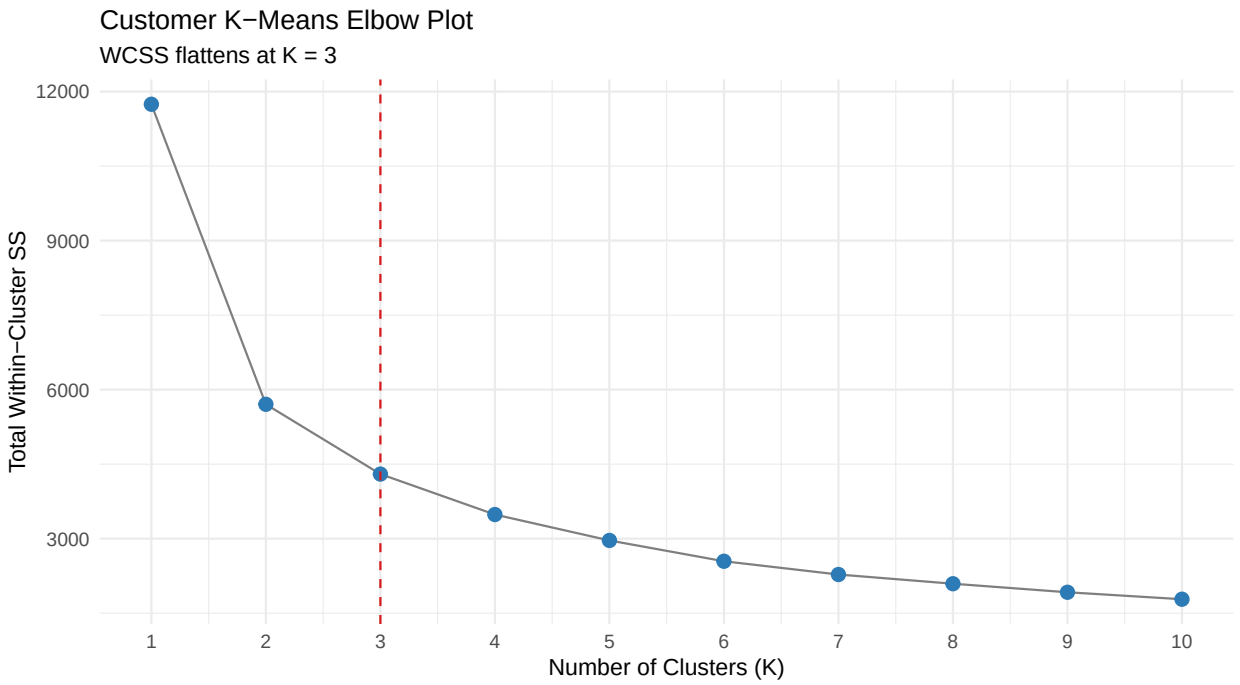


All three RFM features are right-skewed. Without transformation, the top spenders and bulk buyers would pull cluster centroids far from the median customer, making the segments useless for most of the base.

```
rfm_transformed <- rfm_data %>%
  mutate(across(c(Recency, Frequency, Monetary), log, .names = "Log_{.col}")) %>%
  mutate(across(starts_with("Log_"), ~ as.vector(scale(.)), .names = "Scale_{.col}"))
```

4.2 Elbow Plot

```
set.seed(42)
matrix_for_kmeans <- rfm_transformed %>%
  select(Scale_Log_Recency, Scale_Log_Frequency, Scale_Log_Monetary)

wcss <- map_dbl(1:10, ~ kmeans(matrix_for_kmeans, centers = .x, nstart = 25, iter.max = 50)$tot.withinss

tibble(K = 1:10, WCSS = wcss) %>%
  ggplot(aes(x = K, y = WCSS)) +
  geom_line(colour = "grey50") +
  geom_point(size = 3, colour = "#2c7bb6") +
  geom_vline(xintercept = 3, linetype = "dashed", colour = "#d7191c") +
  scale_x_continuous(breaks = 1:10) +
  labs(title = "Customer K-Means Elbow Plot",
       subtitle = "WCSS flattens at K = 3",
       x = "Number of Clusters (K)", y = "Total Within-Cluster SS") +
  theme_minimal(base_size = 12)
```




```
set.seed(42)
customer_kmeans <- kmeans(matrix_for_kmeans, centers = 3, nstart = 25, iter.max = 50)

rfm_segmented <- rfm_transformed %>%
  mutate(Cluster = as.factor(customer_kmeans$cluster))

cluster_profile <- rfm_segmented %>%
  group_by(Cluster) %>%
  summarise(
    Customers     = n(),
    Avg_Recency   = round(mean(Recency), 1),
    Avg_Frequency = round(mean(Frequency), 1),
    Avg_Monetary  = round(mean(Monetary), 2),
    .groups = "drop"
  )

cluster_profile
```

```
## # A tibble: 3 x 5
##   Cluster Customers Avg_Recency Avg_Frequency Avg_Monetary
##   <fct>       <int>       <dbl>         <dbl>        <dbl>
## 1 1            1522        51.9           3.6        1290.
## 2 2            1700       162.            1.3         332.
## 3 3             694        13.7          12.9        6828.
```

4.3 Why This Isn't Enough

The customer model gives us a clean 3-way split, but it groups accounts by total spend — it says nothing about *what* those customers are buying or what that means physically for inventory.

A B2B wholesaler ordering 2,000 cheap bags in a single invoice and a boutique buyer picking up 2 premium items can score identically on Monetary. But one needs pallet storage and cross-docking, the other needs a secure shelf. **The RFM model can't tell them apart.** This is why the analysis pivots to the product axis.

---

5. Model 2 — Product Behavioral Segmentation

```
product_matrix <- retail_data %>%
  mutate(
    StockCode       = tolower(StockCode),
    nChar_StockCode = str_length(StockCode)
  ) %>%
  filter(Country == "United Kingdom", !is.na(CustomerID),
         Quantity > 0, UnitPrice > 0,
         nChar_StockCode %in% c(5, 6, 7) | str_detect(StockCode, "dcgs")) %>%
  mutate(Line_Total = Quantity * UnitPrice) %>%
  group_by(StockCode) %>%
  summarise(
    Total_Quantity = sum(Quantity),
    Total_Revenue  = sum(Line_Total),
    Unique_Buyers  = n_distinct(CustomerID),
    Avg_Price      = mean(UnitPrice),
    .groups = "drop"
  )
```

Each product gets four features:

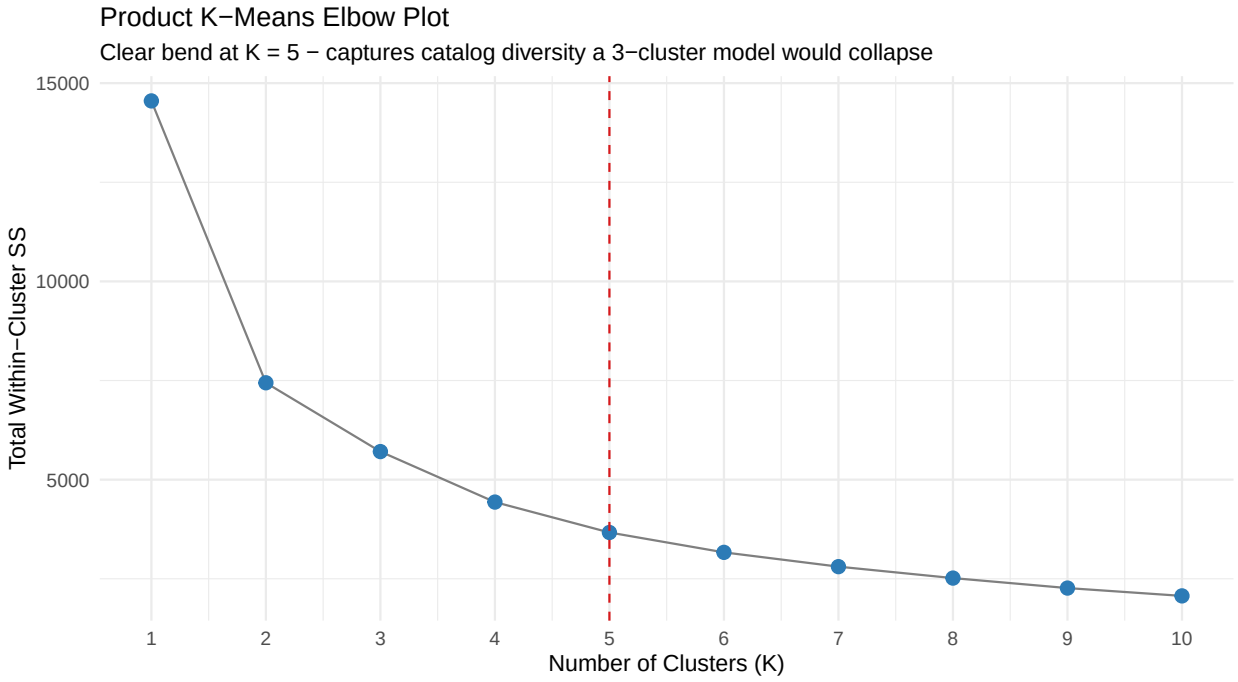
- **Total_Quantity** — how fast it moves through the warehouse
- **Total_Revenue** — its gross financial contribution
- **Unique_Buyers** — wide retail appeal vs. niche bulk purchasing
- **Avg_Price** — which pricing tier it sits in

```
product_transformed <- product_matrix %>%
  mutate(across(c(Total_Quantity, Total_Revenue, Unique_Buyers, Avg_Price),
                log, .names = "Log_{.col}")) %>%
  mutate(across(starts_with("Log_"),
                ~ as.vector(scale(.)), .names = "Scale_{.col}"))

product_kmeans_matrix <- product_transformed %>%
  select(starts_with("Scale_"))
```

5.1 Elbow Plot

```
set.seed(42)
product_wcss <- map_dbl(1:10, ~ {
  kmeans(product_kmeans_matrix, centers = .x, nstart = 25, iter.max = 100)$tot.withinss
})
```

```r
tibble(K = 1:10, WCSS = product_wcss) %>%
  ggplot(aes(x = K, y = WCSS)) +
  geom_line(colour = "grey50") +
  geom_point(size = 3, colour = "#2c7bb6") +
  geom_vline(xintercept = 5, linetype = "dashed", colour = "#d7191c") +
  scale_x_continuous(breaks = 1:10) +
  labs(title = "Product K-Means Elbow Plot",
       subtitle = "Clear bend at K = 5 - captures catalog diversity a 3-cluster model would collapse",
       x = "Number of Clusters (K)", y = "Total Within-Cluster SS") +
  theme_minimal(base_size = 12)
```



5.2 Final Product Clusters

```r
set.seed(42)
final_product_kmeans <- kmeans(product_kmeans_matrix, centers = 5, nstart = 25, iter.max = 100)

product_segmented <- product_matrix %>%
  mutate(Product_Cluster = as.factor(final_product_kmeans$cluster))

product_profiles <- product_segmented %>%
  group_by(Product_Cluster) %>%
  summarise(
    SKU_Count   = n(),
    Avg_Volume  = round(mean(Total_Quantity), 1),
    Avg_Revenue = round(mean(Total_Revenue), 2),
    Avg_Buyers  = round(mean(Unique_Buyers), 1),
    Avg_Price   = round(mean(Avg_Price), 2),
    .groups = "drop"
```

```
  ) %>%
  arrange(desc(Avg_Revenue))

product_profiles
```

```
## # A tibble: 5 x 6
##   Product_Cluster SKU_Count Avg_Volume Avg_Revenue Avg_Buyers Avg_Price
##   <fct>               <int>      <dbl>       <dbl>      <dbl>     <dbl>
## 1 3                     745     3554.       7480.       184.       3.07
## 2 2                     797      218        1017.        38.4      6.91
## 3 1                     958     1429.        843.        65.2      0.87
## 4 4                     591       92.1        85.8       10        1.2
## 5 5                     548        9.1        43.8        3.4      5.27
```

5.3 Cluster Visualization

```
# Replace the bubble chart chunk with this

archetype_labels <- c(
  "1" = "Core Drivers",
  "2" = "Slow Essentials",
  "3" = "Premium Items",
  "4" = "Bulk Commodities",
  "5" = "Dead Stock"
)

# Adjust the key above to match your actual cluster numbers from product_profiles

product_profiles %>%
  mutate(Archetype = archetype_labels[as.character(Product_Cluster)]) %>%
  select(Archetype, Avg_Volume, Avg_Revenue, Avg_Buyers, Avg_Price) %>%
  pivot_longer(-Archetype, names_to = "Metric", values_to = "Value") %>%
  mutate(
    Metric = recode(Metric,
      "Avg_Volume"  = "Avg Units Sold",
      "Avg_Revenue" = "Avg Revenue (£)",
      "Avg_Buyers"  = "Avg Unique Buyers",
      "Avg_Price"   = "Avg Unit Price (£)"
    ),
    Archetype = fct_reorder(Archetype, Value, .fun = max)
  ) %>%
  ggplot(aes(x = Archetype, y = Value, fill = Archetype)) +
  geom_col(show.legend = FALSE, width = 0.65) +
  geom_text(aes(label = comma(round(Value, 1))),
            hjust = -0.1, size = 3.2, colour = "grey30") +
  facet_wrap(~Metric, scales = "free_x", nrow = 2) +
  coord_flip() +
  scale_y_continuous(expand = expansion(mult = c(0, 0.25)),
                     labels = comma) +
  scale_fill_brewer(palette = "Set2") +
  labs(
```

```
    title = "Product Cluster Profiles Across Four Metrics",
    subtitle = "Each panel shows how the five archetypes differ on a single dimension",
    x = NULL, y = NULL
  ) +
  theme_minimal(base_size = 12) +
  theme(
    strip.text       = element_text(face = "bold"),
    panel.grid.major.y = element_blank(),
    panel.grid.minor   = element_blank()
  )
```

---

6. Inventory Action Matrix

Archetype	Avg Units	Avg Revenue	Avg Price	Buyers	Strategy
Core Drivers	~3,554	~£7,480	—	~184	Automate safety stock; fast-pick lanes
Bulk Commodities	~1,429	—	~£0.87	~65	Cross-dock; bundle promotions
Premium Items	~218	~£1,017	~£6.91	—	Price-protect; secure storage; VIP lists

Archetype	Avg Units	Avg Revenue	Avg Price	Buyers	Strategy
Slow Essentials	~92	—	~£1.20	~10	Checkout add-ons; no aggressive reorder
Dead Stock	~9	—	~£5.27	~3	Halt procurement; immediate clearance

---

7. Validation

7.1 ANOVA — Are the Clusters Statistically Real?

```
anova_qty  <- aov(Total_Quantity ~ Product_Cluster, data = product_segmented)
anova_rev  <- aov(Total_Revenue  ~ Product_Cluster, data = product_segmented)
anova_prc  <- aov(Avg_Price      ~ Product_Cluster, data = product_segmented)

map(list(Volume = anova_qty, Revenue = anova_rev, Price = anova_prc), summary)
```

```
## $Volume
##                   Df    Sum Sq   Mean Sq F value Pr(>F)
## Product_Cluster    4 6.445e+09 1.611e+09   195.6 <2e-16 ***
## Residuals       3634 2.994e+10 8.239e+06
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## $Revenue
##                   Df    Sum Sq   Mean Sq F value Pr(>F)
## Product_Cluster    4 2.869e+10 7.172e+09   284.6 <2e-16 ***
## Residuals       3634 9.157e+10 2.520e+07
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## $Price
##                   Df Sum Sq Mean Sq F value Pr(>F)
## Product_Cluster    4  20829    5207   121.5 <2e-16 ***
## Residuals       3634 155720      43
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Here is what each column means:

- **Df** — degrees of freedom. The cluster row will always show `K - 1` (here, 4 for 5 clusters). The residuals row is everything else.
- **Sum Sq / Mean Sq** — how much variance is explained *between* clusters vs. left unexplained *within* them. A large between/within ratio is what you want.

- **F value** — the ratio of between-cluster variance to within-cluster variance. The higher this number, the more separated the clusters are.
- **Pr(>F)** — the p-value. This is the probability of seeing this much separation by random chance. Values below 0.05 are considered significant.

What the asterisks mean:

Symbol	p-value threshold	Plain English
.	$p < 0.10$	Weak signal
*	$p < 0.05$	Significant
**	$p < 0.01$	Strongly significant
***	$p < 0.001$	Extremely significant

All three tests here return ***, meaning the probability that these cluster differences happened by random chance is less than 0.1%. The clusters are real — they represent genuinely distinct product groups, not arbitrary cuts through a uniform distribution.

The --- you sometimes see at the bottom of the output is just R's significance legend label for the scale itself, not a result.

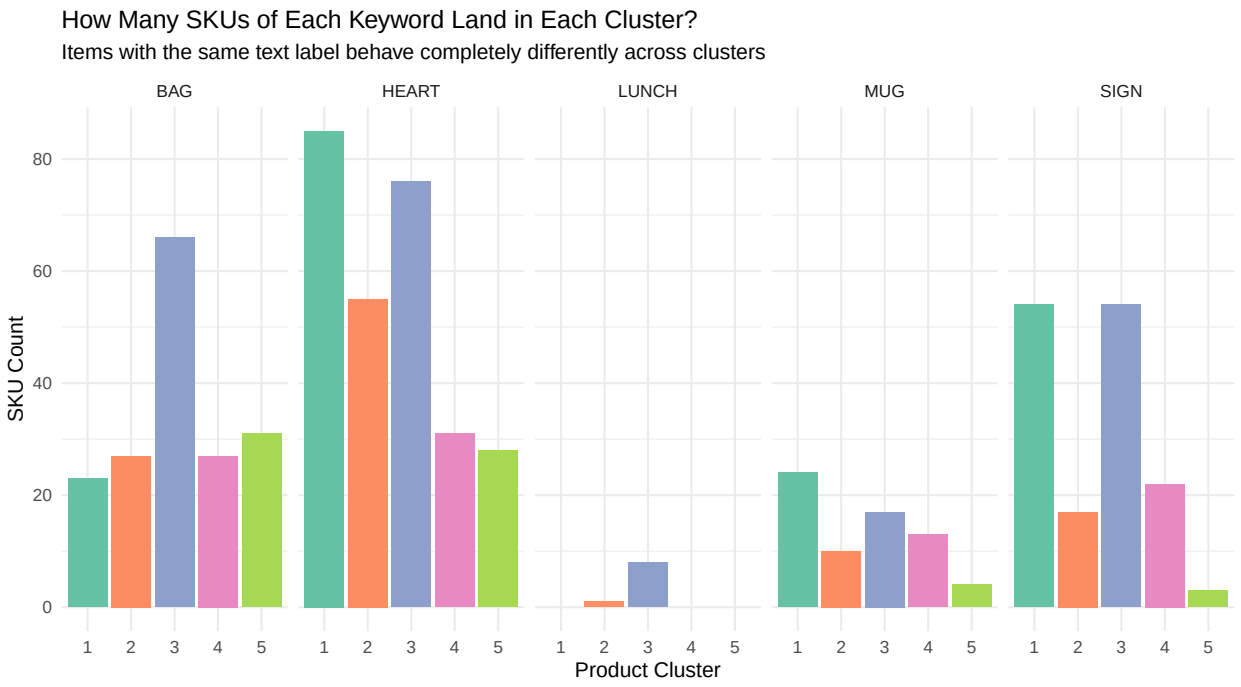
7.2 Semantic Audit — Does "BAG" Mean the Same Thing Everywhere?

```
catalog_lookup <- cleaned_retail %>%
  distinct(StockCode, Description) %>%
  group_by(StockCode) %>% slice(1) %>% ungroup()

product_validation_universe <- product_segmented %>%
  left_join(catalog_lookup, by = "StockCode") %>%
  mutate(Description_Clean = toupper(coalesce(Description, "UNKNOWN")))

target_keywords <- c("MUG", "BAG", "HEART", "LUNCH", "SIGN")

semantic_audit <- product_validation_universe %>%
  filter(str_detect(Description_Clean, paste(target_keywords, collapse = "|"))) %>%
  mutate(Keyword = case_when(
    str_detect(Description_Clean, "MUG")   ~ "MUG",
    str_detect(Description_Clean, "BAG")   ~ "BAG",
    str_detect(Description_Clean, "HEART") ~ "HEART",
    str_detect(Description_Clean, "LUNCH") ~ "LUNCH",
    str_detect(Description_Clean, "SIGN")  ~ "SIGN",
    TRUE ~ "OTHER"
  )) %>%
  group_by(Keyword, Product_Cluster) %>%
  summarise(
    SKUs       = n(),
    Avg_Volume = round(mean(Total_Quantity), 1),
    Avg_Revenue = round(mean(Total_Revenue), 2),
    Avg_Price   = round(mean(Avg_Price), 2),
    .groups    = "drop"
  ) %>%
  arrange(Keyword, Product_Cluster)
```

```
semantic_audit
```

```
## # A tibble: 22 x 6
##    Keyword Product_Cluster  SKUs Avg_Volume Avg_Revenue Avg_Price
##    <chr>   <fct>           <int>      <dbl>       <dbl>     <dbl>
##  1 BAG     1                  23     1329.       1024.      1.03
##  2 BAG     2                  27      168.        622.      5.18
##  3 BAG     3                  66     6216.      10666.      1.91
##  4 BAG     4                  27       67.1        89.9     1.5
##  5 BAG     5                  31       10.7        45.0     4.14
##  6 HEART   1                  85     1454.        966.      0.97
##  7 HEART   2                  55      214.        985.      6.42
##  8 HEART   3                  76     3384.       7511.      2.88
##  9 HEART   4                  31      102.        120.      1.25
## 10 HEART   5                  28        8.4        42.4     4.98
## # i 12 more rows
```

```
semantic_audit %>%
  ggplot(aes(x = Product_Cluster, y = SKUs, fill = Product_Cluster)) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~Keyword, nrow = 1) +
  scale_fill_brewer(palette = "Set2") +
  labs(
    title = "How Many SKUs of Each Keyword Land in Each Cluster?",
    subtitle = "Items with the same text label behave completely differently across clusters",
    x = "Product Cluster", y = "SKU Count"
  ) +
  theme_minimal(base_size = 11)
```



The BAG audit alone makes the case clearly:

- **66 "BAG" SKUs** sit in the Core Drivers cluster — averaging £10,666 revenue and 6,216 units
- **31 "BAG" SKUs** sit in Dead Stock — averaging 10.7 units sold all year at £4.14 each

A warehouse applying a uniform reorder rule to all lines labeled "BAG" would chronically under-stock its top earners while burying capital in dead-weight inventory. The behavioral model separates these automatically.

---

8. Conclusion

The central claim of this analysis is straightforward: **product text labels and customer RFM scores are both insufficient as the primary input to inventory strategy.**

Text labels ignore commercial performance entirely. RFM scores collapse product-level differences into a single monetary total. Neither framework tells you which SKUs to prioritize on the warehouse floor, which ones to protect with safety stock, and which ones are bleeding capital on the shelf right now.

The behavioral product segmentation built here gives you that — five clean archetypes derived from how products actually perform, not what they're called or who bought them. The ANOVA confirms the clusters are statistically distinct. The semantic audit confirms they capture real operational differences that text labels hide.

The next natural step is automating this classification as a periodic refresh — re-running the clustering monthly or quarterly as the catalog evolves, and feeding cluster assignments directly into replenishment rules.

Note

AI assistance was used for guided learning and drafting support. Analysis, implementation, and modeling decisions are the author's own work.